

Python

Summary Session 3



Преподаватель

Портрет

Имя Фамилия

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)

Важно

- 

Камера должна быть включена на протяжении всего занятия
- 

В течение занятия вопросы задавать в чате или когда преподаватель спрашивает, есть ли у Вас вопросы
- 

Вести себя уважительно и этично по отношению к остальным участникам занятия
- 

Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях
- 

Во время занятия будут интерактивные задания, будьте готовы включить камеру или демонстрацию экрана по просьбе преподавателя

Экспресс-опрос

?

Что такое операторы ветвления?

?

Какие существуют ключевые слова для операторов ветвления?



Операторы ветвления

Это конструкции, которые позволяют выполнять различные блоки кода в зависимости от выполнения условий.

Основные операторы ветвления



if — проверяет условие, и если оно истинно, выполняет блок кода.



elif (сокращение от "else if") — проверяет дополнительное условие, если предыдущие были ложными, выполняет блок кода если истинно.



else — выполняет блок кода, если все предыдущие условия оказались ложными.



Ключевые слова

Это зарезервированные слова, которые имеют специальное значение и используются для выполнения определённых операций или действий.

Ключевые слова



if — проверяет условие, и если оно истинно, выполняет блок кода.



elif (сокращение от "else if") — проверяет дополнительное условие, если предыдущие были ложными, выполняет блок кода если истинно.



else — выполняет блок кода, если все предыдущие условия оказались ложными.

Экспресс-опрос

?

Что такое оператор if?

?

Из чего состоит синтаксис оператора if?



Оператор if

Это один из ключевых операторов ветвления в Python, который позволяет программе выполнять определённый блок кода (тело), если заданное условие является истинным (True).

Синтаксис оператора if:



if условие:

- # начало блока кода или тела
- # код, который выполнится, если условие истинно
- # код может занимать любое количество строк, пока не закончится отступ
- # конец блока кода или тела
- # код на этой строке не является частью блока if

Синтаксис оператора if



Определение

Условие — это выражение, которое может быть истинным (True) или ложным (False).

Если условие истинно, выполняется блок кода под оператором if. Если ложно — программа пропускает этот блок.

Код

```
x = 10
if x > 5:
    print("x больше 5")    # выполнится

y = 1
if y == 0:
    print("y равняется 1") # не выполнится
```

Экспресс-опрос

?

Что такое неявное преобразование в bool?

Неявное преобразование в bool



Неявное преобразование в bool происходит в конструкциях, где ожидается логический тип данных. К примеру это происходит в контексте операторов ветвления.

Неявное преобразование в bool



True

```
a = 5
if a: # Неявное преобразование числа 5 из
    # переменной a в True
    print("Число 5 интерпретируется как True")
```

False

```
s = ""
if s: # Неявное преобразование пустой строки в
    # False
    print("Пустая строка интерпретируется как
    False")
```

Экспресс-опрос

?

Что такое оператор elif?

?

Из чего состоит синтаксис оператора elif?



Оператор elif

Используется для проверки дополнительных условий в конструкции ветвления после оператора if.

Синтаксис оператора elif



```
if условие1: # Конструкция обязательно должна начинаться с оператора if
    # Выполняется, если условие1 истинно.
elif условие2:
    # Выполняется, если условие1 ложно, а условие2 истинно
elif условие3:
    # Выполняется, если предыдущие условия ложны, а условие2 истинно
...
```

Пример



```
x = 7
```

```
if x > 10:
```

```
    print("x больше 10")
```

```
elif x > 5:
```

```
    print("x больше 5, но меньше или равно 10")
```

Синтаксис оператора elif



```
age = 25
if age < 18:
    print("Несовершеннолетний")
elif age < 35:
    print("Молодой взрослый")
elif age < 60:
    print("Взрослый")
```



Оператор else

Используется вместе с операторами if и elif для выполнения кода, если предыдущие условия были ложными.

Синтаксис оператора else



```
if условие: # Конструкция обязательно должна начинаться с оператора if
    # Выполняется, если условие истинно (True)
elif условие2: # Оператор elif не обязателен
    # Выполняется, если условие1 ложно, а условие2 истинно
else:
    # Выполняется, если условие ложно (False)
```

Пример



```
x = 3
```

```
if 1 < x < 5:
```

```
    print("x между 1 и 5")
```

```
else:
```

```
    print("x не между 1 и 5")
```

Пример



```
age = 25
if age < 18:
    print("Несовершеннолетний")
elif age < 30:
    print("Молодой взрослый")
elif age < 50:
    print("Взрослый")
else:
    print("Пожилый")
```


Экспресс-опрос

?

Для чего может использоваться несколько операторов if?



Несколько операторов if

Позволяет использовать несколько независимых операторов if, каждый из которых будет проверять своё условие и выполнять соответствующий блок кода, если условие истинно.

Синтаксис с несколькими if:



```
if условие1:
```

```
    # Выполняется, если условие1 истинно
```

```
if условие2:
```

```
    # Выполняется, если условие2 истинно
```

```
if условие3:
```

```
    # Выполняется, если условие3 истинно
```

Пример



```
x = 7
```

```
if x > 5:
```

```
    print("x больше 5") # Это условие истинно, выполнится
```

```
if x < 10:
```

```
    print("x меньше 10") # Это условие тоже истинно, выполнится
```

```
if x == 7:
```

```
    print("x равно 7") # Это условие также истинно, выполнится
```

Экспресс-опрос

?

Что такое вложенные условия?

?

Из чего состоит синтаксис вложенных условий?



Вложенные условия

Это конструкции, в которых один оператор `if` находится внутри другого оператора `if`. Это позволяет проверять дополнительные условия, если первоначальное условие оказалось истинным.

Синтаксис вложенных условий



```
if условие1:
    if условие2:
        # код, который выполнится, если оба условия истинны
    else:
        # код, который выполнится, если условие1 истинно, а условие2 ложно
else:
    # код, который выполнится, если условие1 ложно
```

Проверка типа данных



```
data = "numbers"
integers = True
if data == "numbers":
    print("Это числовые данные.")
elif data == "integers":
    print("Числа целые.")
else:
    print("Числа с плавающей запятой.")
else:
    print("Это нечисловые данные.")
```


Условия для отпуска



```
vacation_requested = True
```

```
available_days = 5
```

```
required_days = 7
```

```
if vacation_requested:
```

```
    print("Запрос на отпуск получен.")
```

```
    if available_days >= required_days:
```

```
        print("Запрос на отпуск одобрен.")
```

```
    else:
```

```
        print("Недостаточно доступных дней для отпуска.")
```

```
else:
```

```
    print("Отпуск не запрашивался.")
```

Проверка наличия данных



```
name_filled = True  
email_filled = True  
email_valid = False
```

```
if name_filled:  
    print("Имя заполнено.")  
    if email_filled:  
        print("Email заполнен.")  
        if email_valid:  
            print("Email действителен.")  
        else:  
            print("Email недействителен.")  
    else:  
        print("Email не заполнен.")  
else:  
    print("Имя не заполнено.")
```

Равносильный код без вложенности



```
name_filled = True
email_filled = True
email_valid = False
if not name_filled:
    print("Имя не заполнено.")
elif not email_filled:
    print("Имя заполнено.")
    print("Email не заполнен.")
elif not email_valid:
    print("Имя заполнено.")
    print("Email заполнен.")
    print("Email недействителен.")
else:
    print("Имя заполнено.")
    print("Email заполнен.")
    print("Email действителен.")
```

Экспресс-опрос

?

Что такое тернарный оператор?

?

В чём заключаются преимущества тернарного оператора?



Тернарный оператор



Тернарный оператор

Является краткой формой записи оператора if-else, когда нужно присвоить значение или выполнить действие в зависимости от условия.

Полезно знать



Без оператора

```
age = 18
```

```
status = "Взрослый" if age >= 18 else  
"Несовершеннолетний"  
print(status)
```

С оператором

```
x = 10
```

```
print("Чётное" if x % 2 == 0 else "Нечётное")
```

Преимущества тернарного оператора:



Компактность: тернарный оператор позволяет сократить запись условий и сделать код более лаконичным, особенно когда нужно присвоить значение на основе простого условия.



Читаемость: для простых проверок тернарный оператор делает код более читаемым, так как вся логика сосредоточена на одной строке.



Использование в выражениях: Тернарный оператор можно использовать внутри других выражений, что позволяет писать сложные конструкции в более сжатом виде.

Экспресс-опрос

?

Что такое конструкция match-case?

?

Из чего состоит синтаксис конструкции match-case?



Конструкция match-case

Конструкция значительно упрощает работу с множеством условий, особенно когда требуется проверить одно значение на соответствие нескольким возможным вариантам.

Синтаксис конструкции match-case:



```
match <выражение>:
    case <шаблон1>:
        # код для выполнения, если выражение соответствует шаблону1
    case <шаблон2>:
        # код для выполнения, если выражение соответствует шаблону2
    case _:
        # код для выполнения, если не совпал ни один шаблон (аналог default)
```

Пример

```
number = 7

match number:
    case 1:
        print("Это один.")
    case 2 | 3: # Символ "|" является аналогом оператора "or"
        print("Это два или три.")
    case _ if number > 5:
        print("Число больше 5.")
    case _:
        print("Это число меньше или равно 5.")
```



Работа с типами данных



```
value = "Hello"

match value:
    case int():
        print("Это целое число.")
    case str():
        print("Это строка.")
    case bool():
        print("Это логический тип.")
    case _:
        print("Неизвестный тип данных.")
```

Здесь проверяется тип данных переменной value, и соответствующий блок кода выполняется в зависимости от типа.

Особенности конструкции match-case:



Четкость и лаконичность: конструкция match-case позволяет избегать множества операторов if-elif, делая код более лаконичным.



Поддержка шаблонов: можно использовать разные типы данных и шаблоны для проверки, включая не только значения, но и условия (case _ if).



Обработка нескольких вариантов через |: можно указать несколько вариантов в одном блоке case, разделяя их с помощью символа |.

Экспресс-опрос

?

Что такое встроенные математические функции?

?

Перечислите существующие математические функции.

Встроенные математические функции



Python предоставляет ряд встроенных математических функций, которые упрощают выполнение базовых вычислений и работы с числами. Эти функции не требуют дополнительного импорта и доступны по умолчанию.

abs()



Определение

Функция `abs()` возвращает абсолютное значение числа (модуль), то есть его значение без знака.

Код

```
print(abs(-7)) # Результат: 7
```

```
print(abs(3.5)) # Результат: 3.5
```

max()



Определение

Функция `max()` возвращает наибольшее из переданных чисел или элементов коллекции.

Код

```
a = 1
b = 5
c = 3
print(max(a, b, c))      # Передаем несколько
                          значений
```

min()



Определение

Функция `min()` возвращает наименьшее из переданных чисел или элементов списка.

Код

```
print(min(1, 5, 3))  
несколько значений
```

Передаем

pow()



Определение

Функция `pow()` принимает два аргумента: число и степень, в которую нужно возвести число.

Код

```
print(pow(2, 3)) # Передаем сначала
                  число, а после степень

print(pow(5, 2))
```

pow()



Определение

Функция `pow()` принимает два аргумента: число и степень, в которую нужно возвести число.

Код

```
print(pow(2, 3)) # Передаем сначала
                  число, а после степень

print(pow(5, 2))
```

sum()



Определение

Функция `sum()` возвращает сумму всех элементов коллекции.

Код

```
print(sum([1, 2, 3, 4]))    # Передаем
набор чисел в квадратных скобках
```

round()



Определение

Функция round() округляет число до ближайшего целого или до указанного количества знаков после запятой.

Код

```
print(round(4.56))    # Округление до  
целого
```

```
print(round(4.567, 2)) # Округление до  
указанного количества знаков)
```

Банковское округление



Определение

По умолчанию, `round()` округляет число до ближайшего целого.

Если десятичная часть числа меньше 0.5, округление идёт в меньшую сторону, если больше 0.5 — в большую.

Код

```
print(round(3.4)) # Округление в  
меньшую сторону
```

```
print(round(3.6)) # Округление в  
большую сторону
```


Банковское округление



Определение

Для случаев, когда десятичная часть точно равна 0.5 Python использует банковское округление.

Это правило заключается в том, что числа округляются в ближайшее чётное целое.

Код

```
print(round(1.5)) # Результат: 2 (округление к  
ближайшему чётному)
```

```
print(round(2.5)) # Результат: 2 (округление к  
ближайшему чётному)
```

```
print(round(3.5)) # Результат: 4 (округление к  
ближайшему чётному)
```

Банковское округление



Определение

Функция `round()` работает аналогично с отрицательными числами.

Правила те же: числа округляются к ближайшему целому или к ближайшему чётному числу в случае 0.5.

Код

```
print(round(-1.5)) # Результат: -2
(округление к ближайшему чётному)
```

```
print(round(-2.5)) # Результат: -2
(округление к ближайшему чётному)
```

Экспресс-опрос

?

Что такое модуль?

?

Зачем нужны модули?



Модуль

Это файл с расширением `.py`, который содержит код на языке Python: функции, классы, переменные, и даже исполняемые инструкции.

Зачем нужны модули?



Организация кода: Модули помогают разбить большие программы на более мелкие, логически связанные части, что упрощает поддержку и понимание кода.



Повторное использование: Модуль можно импортировать в несколько разных программ, не переписывая код каждый раз заново.



Изоляция кода: Модули помогают изолировать код, что позволяет избежать конфликтов имён функций, классов и переменных.

Экспресс-опрос

?

Что такое оператор import?

?

Как осуществить импорт модуля с псевдонимом?



Оператор import

Модули в Python можно импортировать в программу, используя ключевое слово `import`. Это позволяет использовать в текущей программе функционал, написанный в других модулях, без необходимости переписывать его заново.

Импорт всего модуля



Определение

Самый простой способ использования оператора `import` — это импортировать весь модуль.

Код

```
import math

print(math.sqrt(16)) # Используем функцию
sqrt() из модуля math
```


Импорт конкретных функций или классов



Определение

Можно импортировать конкретные функции или классы из модуля с помощью ключевого слова `from`.

Код

```
from math import sqrt, pi

print(sqrt(16)) # Используем функцию sqrt() без
                # указания имени модуля

print(pi)       # Используем константу pi
```

Импорт модуля с псевдонимом



Определение

Если название модуля длинное или если вы хотите использовать более короткое имя, можно задать псевдоним для модуля с помощью ключевого слова `as`.

Код

```
import math as m

print(m.sqrt(16)) # Используем функцию sqrt()
из модуля math, но с псевдонимом "m"
```

Импорт всего содержимого модуля



Определение

Можно импортировать всё содержимое модуля в текущую область видимости с помощью оператора `*`.

Код

```
from math import *  
  
print(sqrt(16)) # Используем функцию sqrt()  
               напрямую  
  
print(pi)      # Используем константу pi  
               напрямую
```

Экспресс-опрос

?

Что такое стандартная библиотека Python?

?

Зачем нужны модули?



Стандартная библиотека Python

Это набор модулей и пакетов, которые включены в Python по умолчанию и предоставляют широкий спектр функциональности для выполнения различных задач.

Зачем нужны модули?



os — взаимодействие с операционной системой



sys — доступ к переменным и функциям, связанным с интерпретатором Python



math — математические функции



datetime — работа с датами и временем



json — работа с JSON-форматом



re — работа с регулярными выражениями



random — генерация случайных чисел

Экспресс-опрос

?

Что такое модуль math?

?

Какие существуют основные функции модуля math?



Модуль math

Это стандартный модуль, предоставляющий функции для выполнения различных математических операций, таких как вычисление тригонометрических функций, логарифмов, возведение в степень, и многое другое.

Как использовать модуль math?



```
import math
```

Основные функции модуля math



Квадратный корень

```
x = 16

result = math.sqrt(x)

print("Квадратный корень из", x, "равен",
      result)
```

Округление числа

```
import math

print(math.ceil(4.3)) # Округление вверх: 5

print(math.floor(4.8)) # Округление вниз: 4
```

Основные функции модуля math



Вычисление факториала

```
n = 5

factorial_result = math.factorial(n)

print("Факториал числа", n, "равен",
      factorial_result)
```

Константы в модуле math

math.pi: Число Пи ($\pi \approx 3.14159$)

math.e: Основание натурального логарифма
($e \approx 2.71828$)

Экспресс-опрос

?

Для чего требуется хранение чисел в памяти?

?

Что такое бит и байт?



Биты и байты

Компьютеры работают с данными в виде битов и байтов. Эти единицы являются основой для представления информации в памяти.



Бит

Бит — это минимальная единица информации, которая может иметь одно из двух значений: 0 или 1.

Биты представляют данные в двоичной системе счисления, которая лежит в основе всех вычислительных процессов.



Байт

Байт — это набор из 8 битов. Он используется как базовая единица для хранения данных в памяти компьютера.

Например, 1 байт может хранить одно целое число в пределах от 0 до 255 (или от -128 до 127 в случае знаковых чисел).



Хранение чисел в памяти

Числа хранятся в памяти в двоичной системе счисления.

Основные функции модуля math



32-битные целые числа

Занимают 4 байта памяти (32 бита).

Могут хранить значения от
 -2^{31} до $2^{31} - 1$
 (от -2 147 483 648
 до 2 147 483 647).

64-битные целые числа

Занимают 8 байт памяти (64 бита)

Могут хранить значения от
 -2^{63} до $2^{63} - 1$
 (от -9 223 372 036 854 775 808
 до 9 223 372 036 854 775 807).

Целые числа (int)



32-битные целые числа:

- Занимают 4 байта памяти (32 бита).
- Могут хранить значения от -2^{31} до $2^{31} - 1$ (от -2 147 483 648 до 2 147 483 647).



64-битные целые числа:

- Занимают 8 байт памяти (64 бита).
- Могут хранить значения от -2^{63} до $2^{63} - 1$ (от -9 223 372 036 854 775 808 до 9 223 372 036 854 775 807).



Хранение чисел в памяти

Python — это высокоуровневый язык программирования, и одна из его особенностей заключается в том, что целые числа в Python не имеют фиксированного размера.

Вещественные числа (float)

В Python вещественные числа обычно представлены в формате 64-битного числа с плавающей запятой.

- Занимают 8 байт памяти (64 бита).
- В таком формате числа хранятся как комбинация мантиссы и экспоненты.



Пример диапазона для 64-битных чисел с плавающей запятой:

- Примерно от $\pm 10^{-308}$ до $\pm 10^{308}$.



Пример целого числа

Число 42 в двоичной системе записывается
как:
00101010 (в 8-битном формате)

Экспресс-опрос

?

Что такое вещественные числа?

?

Приведите пример вещественного числа.



Вещественные числа

Числа с плавающей запятой (вещественные числа) в Python хранятся по стандарту IEEE 754, который используется большинством современных систем.

Вещественные числа



1 бит — знак числа (положительное или отрицательное).



11 битов — экспонента, которая указывает на степень числа.



52 бита — мантисса, представляющая саму дробную часть числа.

Вещественные числа



Пример

$$0.0001011 = 1.011 * 10^{-4}$$

Здесь

Мантисса: 1.011 (сохраняет значимые цифры числа, избавившись от нулей)

Экспонента: -4 (указывает на степень 10, то есть количество потерянных нулей)



Ограниченная точность

Компьютер может хранить только определённое количество цифр в мантиссе.

В Python это обычно 52 бита для мантиссы (в 64-битной системе).

Погрешность в вычислениях



Пример

Когда мы складываем такие числа, могут возникнуть неточности:

```
print(0.1 + 0.2) # Ожидаем 0.3
```

Результат

0.30000000000000004

Экспресс-опрос

?

Почему возникают такие ошибки?

Почему возникают ошибки

Компьютеры используют двоичную систему, а многие десятичные дроби, такие как 0.1 или 0.2, не могут быть точно представлены в двоичной системе.

Погрешность в вычислениях



Пример

Представление числа в двоичной системе

Число 0.1 в двоичной

0.000110011001100110011...
(бесконечная последовательность)

Сравнение вещественных чисел



Операции сравнения

Из-за этих погрешностей сравнение вещественных чисел может работать не так, как ожидается.

Пример

```
print(0.1 + 0.2 == 0.3) # Вывод: False
```

Сравнение вещественных чисел



Округление результата

Для устранения небольших ошибок при выводе или сравнении результатов можно использовать функцию `round()`, которая округляет вещественные числа до указанного количества знаков после запятой.

Пример

```
a = 0.1 + 0.2

print(round(a, 2))           # Вывод: 0.3

print(round(a, 2) == 0.3)   # Вывод: True
```




Использование модуля decimal

Если требуется высокая точность при работе с вещественными числами, в Python можно использовать модуль `decimal`, который позволяет задавать точность операций и избегать ошибок округления.

Экспресс-опрос

?

Что такое модуль Decimal?



Модуль Decimal

Позволяет представлять числа с плавающей запятой в десятичной системе без потери точности.

Модуль Decimal



```
# Импортируем класс Decimal из модуля decimal

from decimal import Decimal

# Создаём два объекта класса Decimal с помощью строк '0.1' и '0.2'

# Это гарантирует, что числа будут точно представлены в десятичном формате

a = Decimal('0.1')

b = Decimal('0.2')

# Сложение без ошибки округления

c = a + b

print(c) # Вывод: 0.3
```



ВОПРОСЫ





ВОПРОСЫ





ЗАДАНИЕ



Задание

Завершить работу над задачами с практикума.



Задание 1

Напишите программу, которая принимает три числа и определяет, являются ли они упорядоченными по возрастанию.

Пример вывода:

Введите первое число: 3

Введите второе число: 5

Введите третье число: 7

Числа упорядочены по возрастанию.

Задание 2



Напишите программу, которая получит два числа от пользователя и выведет большее из них. Если числа равны, программа должна вывести "Числа равны".

Пример вывода:

Введите первое число: 8

Введите второе число: 8

Числа равны

Задание 3

Напишите программу, которая получит число от пользователя и выведет, что это число: "Четное" или "Нечетное". Решите обычным и тернарным оператором.

Пример вывода:

Введите число: 7

Нечетное

Задание 4



Напишите программу, которая получит от пользователя возраст и выведет соответствующую категорию: ребенок (меньше 12 лет), подросток (от 12 до 18), взрослый (от 18 до 60), пожилой (больше 60).

Пример вывода:

Введите возраст: 25

Вы взрослый.

Задание 5

Напишите программу, которая получит три числа от пользователя и определит, может ли существовать треугольник с такими сторонами. Условие существования треугольника: сумма любых двух сторон должна быть больше третьей.

Пример вывода:

Введите первую сторону: 3

Введите вторую сторону: 4

Введите третью сторону: 5

Такой треугольник может существовать.

Задание 6



Напишите программу, которая принимает строку и проверяет, является ли она пустой.

Пример вывода:

Введите строку:

Строка пуста.

Задание 7

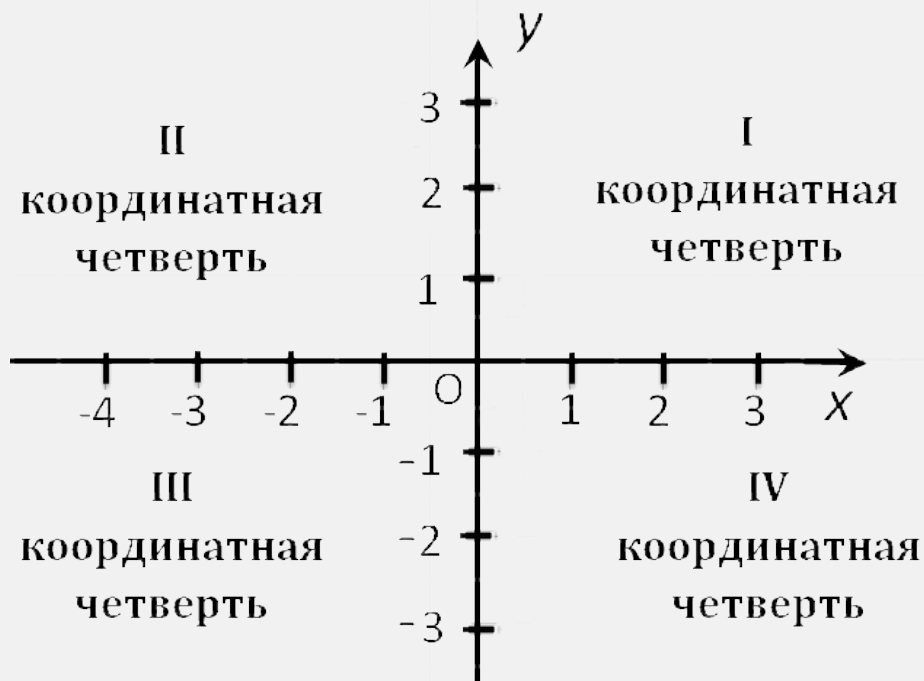
Напишите программу, которая запрашивает координаты точки по осям x и y и определяет, в каком квадранте она находится, либо то что она находится на одной из осей.

Пример вывода:

Введите координату x : -3

Введите координату y : 5

Точка находится во втором квадранте.



Задание 8

Напишите программу, которая запрашивает у пользователя часы и минуты и проверяет, являются ли они допустимыми значениями времени.

Пример вывода:

Введите часы: 25

Введите минуты: 30

Некорректное время.

Задание 9*

Напишите программу, которая принимает три числа и определяет знак их произведения без явного умножения.

Пример вывода:

Введите три числа:

-2

5

-3

Произведение положительное.

Задание 10

Напишите программу, которая рассчитывает налог на доход в зависимости от вводимой суммы и ставок:

- До 10,000 включительно: налог 5%.
- От 10,001 до 50,000 включительно: налог 10%.
- Свыше 50,000: налог 20%.

Обратите внимание, что на сумму выше 10000, налог оплачивается как 5% на 10000 из суммы, а следующие проценты на то что превышает границу.

Программа должна вывести налог и итоговую сумму после уплаты налога.

Пример вывода:

Введите доход: 45000

Налог: 4500.0

Итоговая сумма после уплаты налога: 40500.0

Задание 11

Напишите программу, которая определяет, нужно ли выдать предупреждение об экстремальной погоде.

Пользователь вводит:

- Температуру.
- Скорость ветра.
- Уровень осадков (в мм).

Условия для предупреждения:

- Температура ниже -10°C и скорость ветра выше 20 м/с.
- Температура выше 35°C .
- Уровень осадков выше 50 мм.

Пример вывода:

Введите температуру: -15

Введите скорость ветра: 25

Введите уровень осадков: 5

Предупреждение: экстремальная погода!



ВОПРОСЫ





РАЗБОР ДОМАШНЕГО ЗАДАНИЯ



Разбор домашнего задания

Сортировка чисел

Напишите программу, которая запрашивает у пользователя три числа и выводит их в порядке возрастания, разделенные запятой.

Пример вывода:

Введите первое число: 5

Введите второе число: 2

Введите третье число: 7

Числа в порядке возрастания: 2, 5, 7

Разбор домашнего задания

```
num1 = int(input("Введите первое число: "))  
num2 = int(input("Введите второе число: "))  
num3 = int(input("Введите третье число: "))  
  
if num1 > num2:  
    num1, num2 = num2, num1  
  
if num1 > num3:  
    num1, num3 = num3, num1  
  
if num2 > num3:  
    num2, num3 = num3, num2  
  
print("Числа в порядке возрастания:", num1, num2, num3)
```

Разбор домашнего задания

Високосный год

Напишите программу, которая запрашивает у пользователя год и проверяет, является ли он високосным. Год является високосным, если он делится на 4 без остатка, и в то же время не делится на 100 без остатка. При этом если год делится на 400 без остатка, он тоже является високосным. Выведите соответствующее сообщение на экран с помощью функции `print`.

Пример вывода:

Введите год: 2024

Год является високосным.

Введите год: 1900

Год не является високосным.

Разбор домашнего задания

```
year = int(input("Введите год: "))

if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
    print("Год является високосным.")
else:
    print("Год не является високосным.")
```

Разбор домашнего задания

Математическое округление

Напишите программу, которая принимает число с плавающей точкой и округляет его до целого числа в соответствии с правилами школьной математики, а не банковского округления. Программа должна корректно работать как с положительными, так и с отрицательными числами.

Пример вывода:

Введите	вещественное	число:	2.5
Округленное значение: 3			

Введите	вещественное	число:	-2.5
Округленное значение: -3			

Разбор домашнего задания

```
num = float(input("Введите вещественное число: "))
```

```
if num > 0:
```

```
    rounded_num = int(num + 0.5)
```

```
else:
```

```
    rounded_num = int(num - 0.5)
```

```
print("Округленное значение:", rounded_num)
```

Разбор домашнего задания

Решение с использованием модуля math:

```
import math
```

```
num = float(input("Введите вещественное число: "))
```

```
rounded_num = math.floor(num + 0.5) if num > 0 else math.ceil(num - 0.5)
```

```
print("Округленное значение:", rounded_num)
```

Разбор домашнего задания

Гипотенуза треугольника

Напишите программу, которая запрашивает у пользователя длины двух катетов прямоугольного треугольника и вычисляет длину гипотенузы. Гипотенуза равна квадратному корню из суммы квадратов катетов.

Пример

вывода:

Введите	длину	первого	катета:	3
Введите	длину	второго	катета:	4
Длина гипотенузы: 5.0				

Разбор домашнего задания

```
import math

a = float(input("Введите длину первого катета: "))
b = float(input("Введите длину второго катета: "))
hypotenuse = math.sqrt(a**2 + b**2)

print("Длина гипотенузы:", hypotenuse)
```



ВОПРОСЫ



Заключение

